Neuromorphic Computing and Engineering

PURPOSE-LED PUBLISHING™

PAPER • OPEN ACCESS

Traces propagation: memory-efficient and scalable forward-only learning in spiking neural networks

To cite this article: Lorenzo Pes *et al* 2026 *Neuromorph. Comput. Eng.* **6** 014002

View the article online for updates and enhancements.

You may also like

- STOP: spatiotemporal orthogonal propagation for weight-threshold-leakage synergistic training of deep spiking neural networks
Haoran Gao, Xichuan Zhou, Yingcheng Lin et al.

- BAM-SLDK: biologically inspired attention mechanism with spiking learnable delayed kernel synapses
Mario Chacón-Falcón, Alberto Patiño-Saucedo, Luis Camuñas-Mesa et al.

- Slax: a composable JAX library for rapid and flexible prototyping of spiking neural networks
Thomas M Summe and Siddharth Joshi

This content was downloaded from IP address 88.23.72.76 on 17/04/2026 at 11:33

IOP Publishing Neuromorph. Comput. Eng. 6 (2026) 014002 https://doi.org/10.1088/2634-4386/ae2ef9

NEUROMORPHIC
Computing and Engineering

Check for updates

OPEN ACCESS

RECEIVED
17 September 2025

REVISED
24 November 2025

ACCEPTED FOR PUBLICATION
18 December 2025

PUBLISHED
7 January 2026

Original content from this work may be used under the terms of the Creative Commons Attribution 4.0 licence.

Any further distribution of this work must maintain attribution to the author(s) and the title of the work, journal citation and DOI.

PAPER

Traces propagation: memory-efficient and scalable forward-only learning in spiking neural networks

Lorenzo Pes*, Bojian Yin, Sander Stuijk and Federico Corradi

Electrical Engineering, Eindhoven University of Technology, Eindhoven, The Netherlands
* Author to whom any correspondence should be addressed.

E-mail: l.pes@tue.nl

Keywords: spiking neural networks, forward-only, on-device learning, local learning, fine-tuning

Abstract

Spiking neural networks (SNNs) provide an efficient framework for processing dynamic spatio-temporal signals and for investigating the learning principles underlying biological neural systems. A key challenge in training SNNs is to solve both spatial and temporal credit assignment. The dominant approach for training SNNs is backpropagation through time (BPTT) with surrogate gradients. However, BPTT is in stark contrast with the spatial and temporal locality observed in biological neural systems, and leads to high computational and memory demands, limiting efficient training strategies and on-device learning. Although existing local learning rules achieve local temporal credit assignment by leveraging eligibility traces, they fail to address the spatial credit assignment without resorting to auxiliary layer-wise matrices, which increase memory overhead and hinder scalability, especially on embedded devices. In this work, we propose traces propagation (TP), a forward-only, memory-efficient, scalable, and fully local learning rule that combines eligibility traces with a layer-wise contrastive loss without requiring auxiliary layer-wise matrices. TP outperforms other fully local learning rules on N-MNIST and SHD datasets. On more complex datasets such as DVS-GESTURE and DVS-CIFAR10, TP showcases competitive performance and scales effectively to deeper SNN architectures such as VGG-9, while providing favorable memory scaling compared to prior fully local scalable rules, for datasets with a significant number of classes. Finally, we show that TP is well suited for practical fine-tuning tasks, such as keyword spotting on the Google speech commands dataset, thus paving the way for efficient learning at the edge.

1. Introduction

One of the fundamental quests of bio-inspired machine intelligence is to formalize learning rules that match the efficiency observed in the biological brain. Recently, spiking neural networks (SNNs) have emerged as a computational model that mimics the efficient behavior of biological neurons, offering a promising substrate for exploring neural systems' learning theories. Fundamentally, learning in SNNs entails solving both temporal and spatial credit assignment problems [1, 2]. The former refers to attributing credit to past synaptic activity based on the effect on future output, while the latter refers to determining how local activity affects higher levels of the network hierarchy. These problems have been successfully addressed by back-propagation-through-time (BPTT), leading to state-of-the-art (SOTA) performance in SNNs [3].

Despite its success, BPTT is widely regarded as biologically implausible because it contradicts several fundamental principles of learning observed in biological neural systems. From a spatial perspective, it suffers from *weight-transport* [4] and *update-locking* [5], due to its reliance on the chain rule to distribute a global error across the hierarchy. These problems entail a global synchronization mechanism, require signals to traverse synapses in reverse order, and enforce non-local, sequential updates, contradicting basic principles of neural systems. From a temporal perspective, BPTT requires knowledge of

© 2026 The Author(s). Published by IOP Publishing Ltd

future states of the network, violating the time locality observed in biological systems and incurring a memory cost that scales with the sequence length [6].

To address these limitations, several learning algorithms have been proposed with the aim of improving biological plausibility and computational efficiency. To mitigate the limitation of non-local temporal credit assignment, three-factor learning rules such as E-prop [2], OTTT [7], OSTL [8], and S-TLLR [9] employ eligibility traces, a short-term memory of neural activity, that enables time-local updates. However, they only partially address the limitations of spatial credit assignment. To address both challenges, learning rules such as ETLP [10], OSTTP [11], DECOLLE [12], and TESS [13] combine eligibility traces with DRTP [14] or layer-wise local error losses. Among these, only TESS has been demonstrated in deeper architectures, such as VGG-9, thus paving the way for fully local and scalable learning in SNNs. Despite its novelty and competitive performance, TESS requires the storage and transposition of auxiliary matrices for each layer to compute the local loss that addresses the spatial credit assignment problem. These matrices scale with the number of neurons in each layer and with the number of output neurons, leading to significant storage requirements, in particular in convolutional networks, where the number of neurons per layer dominates the memory footprint.

Recently, a novel class of learning algorithms, known as Forward-Only [15–17], has emerged as a potential solution to the spatial credit assignment problem, without incurring the cost of layer-wise auxiliary matrices. Among these, Signal Propagation (SP) [16] stands out as the only solution with proven scalability to more complex architectures such as VGG-9. However, its applicability in SNNs remains an open question. To address this gap, we propose a novel temporally and spatially local learning rule that unifies forward-only learning with SNNs, enabling scalable and memory efficient on-device training for deep architectures under constrained resources. The main contributions of this work are summarized as follows:

- We propose traces propagation (TP), a novel and scalable fully-local learning rule that unifies eligibility traces for temporal credit assignment with a local contrastive loss for spatial credit assignment, enabling training of deep SNNs in a forward-only manner.
- Unlike previous local learning rules, which require projection matrices at each layer, TP employs a single global projector and a modulation term whose dimensionality can be adapted to the available hardware resources. This removes the depth-dependent projection requirement and leads to a more memory-efficient implementation for deep networks with large output dimensionality.
- TP outperforms all other fully local learning rules on the N-MNIST and Spiking Heidelberg Digit (SHD) datasets, and shows competitive performance on DVS-GESTURE and DVS-CIFAR10 using deep architectures such as VGG-9. Furthermore, we demonstrate its suitability for real-world scenarios through fine-tuning on the Google speech commands (GSC) dataset for keyword spotting, highlighting its applicability to resource-constrained edge devices.

1.1. Notation

Throughout this work, we use the Einstein summation notation with indices enclosed in square brackets (e.g. $[i]$, $[j]$), where repeated indices imply summation over their corresponding dimensions. This convention applies to tensors, and when no index is shown, a scalar quantity is assumed.

1.2. Background & SOTA

1.2.1. Neuron model

Throughout this work, we employ the leaky-integrate-and-fire (LIF) [18], the most common neuron model used to implement SNNs. In discrete time, the neuronal dynamic of the LIF model is governed by the following equations:

$$v_l^t[j] = \alpha_l v_l^{t-1}[j] + s_{l-1}^t[i]\, W_l[i,j] + s_l^{t-1}[j]\, R_l[j,j] - s_l^{t-1}[j]\, v_{\text{th}}, \tag{1}$$

$$s_l^t[j] = \Theta\left(v_l^t[j] - v_{\text{th}}\right). \tag{2}$$

In equation (1), $v_l^t[j]$, represents the membrane potential of neurons in layer l at time step t, $W_l[i,j] \in \mathbb{R}^{H_{l-1}\times H_l}$ the synaptic connections between neurons in layer l and $l-1$, $R_l[j,j] \in \mathbb{R}^{H_l\times H_l}$ the recurrent matrix, $\alpha_l = e^{-\Delta t/\tau_m}$ the membrane decay with time constant τ_m and s_l^t the spike event. In equation (2), v_{th} represents the membrane threshold and Θ the heavy-side step activation function. Due to the discontinuity of the heavy-side step function, a surrogate gradient $\partial s_l^t[j]/\partial v_l^t[j] = \Theta'$ is used to enable gradient-based learning [3].

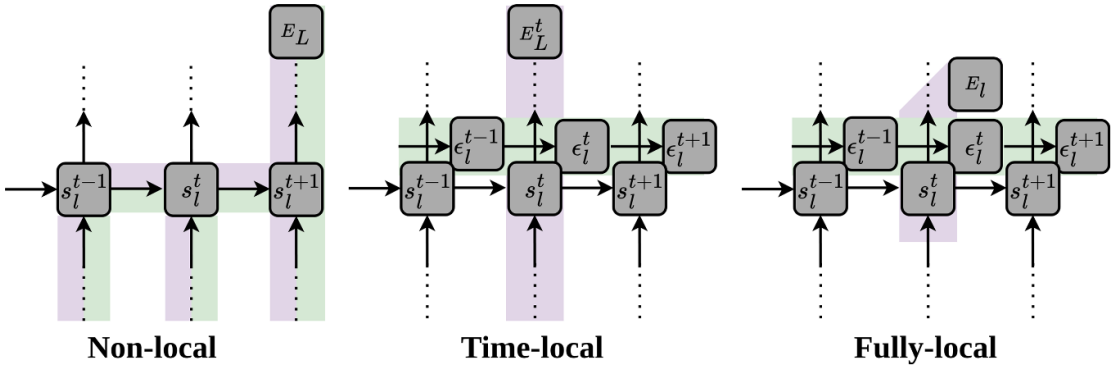

Figure 1. Locality comparison in learning algorithms. Violet represents the spatial credit assignment, while green indicates the temporal credit assignment. *Left:* Non-local learning (e.g. BPTT) relies on a global error signal E_L that requires both spatial and temporal components for the whole sequence. *Center:* Time-local learning (e.g, E-prop, OSTL, OTTP and S-TLLR) use eligibility traces ϵ_l^t to remove temporal dependencies, but still requires a global spatial credit assignment based on E_l^t. *Right:* Fully-local learning (e.g. ETLP, OSTTP,TESS and TP), still uses eligibility traces for temporal credit assignment but uses only local information for the spatial assignment, thus satisfying both temporal and spatial locality.

1.3. Error BPTT & its limitations

From a machine learning perspective, the problem of learning has been addressed using error BPTT, leading to SOTA performance in various tasks of different complexity. The main objective of BPTT is to provide a method to change the learnable parameters $\theta_l = \{W_l[i,j], R_l[j,j]\}$, such that the error at the output layer E_L is minimized. This error is calculated by comparing the activity of neurons in the last layer with an expected activity $(c^*[c])^t$. To minimize this loss, the learnable parameters are generally updated following the gradient descent as

$$\Delta\theta_l[i,j] = -\eta \cdot \frac{\partial E_L}{\partial \theta_l[i,j]} \tag{3}$$

For a network of LIF neurons without recurrent synaptic connections, i.e. $\theta_l = \{W_l[i,j]\}$, there is still a temporal dependency across time steps due to the leaky integration mechanism of the membrane potential. Specifically, the membrane potential $v_l^t[j]$ at time t depends on its value in the previous time step $v_l^{t-1}[j]$, through the decaying factor α_l. Thus, the gradient $\frac{\partial E_L}{\partial \theta_l}$ must account for the influence of each parameter on the entire temporal sequence. To find this gradient, the network is unfolded across the full sequence and BPTT is applied as

$$\left(\frac{\partial E_L}{\partial \theta_l[i,j]}\right)_{\text{BPTT}} = \sum_{t=1}^{T} \frac{\partial E_L}{\partial v_l^t[j]} \frac{\partial v_l^t[j]}{\partial W_l[i,j]} \tag{4}$$

$$= \sum_{t=1}^{T} \left(\underbrace{\frac{\partial E_L}{\partial s_l^t[j]} \frac{\partial s_l^t[j]}{\partial v_l^t[j]}}_{\text{spatial}} + \underbrace{\frac{\partial E_L}{\partial v_l^{t+1}[j]} \frac{\partial v_l^{t+1}[j]}{\partial v_l^t[j]}}_{\text{temporal}} \right) \underbrace{\frac{\partial v_l^t[j]}{\partial W_l[i,j]}}_{\text{spatial}}. \tag{5}$$

From this equation, it is evident that BPTT consists of both a temporal and a spatial component, which are entangled together, as shown in figure 1(a). This entanglement exists because the gradient at each time step recursively depends on the future gradient, making the evaluation of the spatial component dependent on the full temporal history. This non-locality of updates leads to a set of problems, which call into question the bio-plausibility of the algorithm and hinder efficient hardware implementations. From a **spatial** perspective, the main limitations are:

- **Update-locking** [4]: This issue stems from the fact that the gradient $\partial E_L / \partial v_l^t[j]$ in equation (5) is not available until the full forward pass has been completed and the error of the output layer has been propagated backward through the network. This sequential dependency not only leads to idle resources but also presupposes a global synchronization mechanism that collides with the massively parallel and local processing observed in the biological brain. Furthermore, update-locking entails storing each layer's activation for a given time step until the corresponding gradient can be computed, resulting in significant memory overhead, particularly in deep convolutional networks, where activations dominate the memory footprint.

- **Weight-transport [5]:** In equation (5), computing $\partial E_L/\ \partial v_l^t[j]$ requires multiplying the backpropagated error $(\partial E_L/\ \partial v_{l+1}^t)$ by the transpose of the forward weight matrix:

$$\frac{\partial E_L}{\partial v_l^t[j]} = \frac{\partial E_L}{\partial v_{l+1}^t[k]} \frac{\partial v_{l+1}^t[k]}{\partial s_l^t[j]} \frac{\partial s_l^t[j]}{\partial v_l^t[j]} \tag{6}$$

$$= \frac{\partial E_L}{\partial v_{l+1}^t[k]} W_{l+1}^T \frac{\partial s_l^t[j]}{\partial v_l^t[j]} \tag{7}$$

  This implies that the output error must traverse the same network in the reverse direction to perform the synaptic update, a principle that is hard to justify from a biological perspective [19]. Furthermore, transposing matrices in digital systems leads to non-contiguous memory accesses, which increases latency and energy consumption [20, 21].

From a **temporal** perspective, the primary limitation arises from the inherent temporal recurrence of the membrane potential, which results in:

- **Activations history**: As expressed in equation (5), the gradient $\partial E_L/\ \partial v_l^t[j]$ recursively depends on future terms via $\partial v_l^{t+1}[k]/\ \partial v_l^t[j]$. This dependency requires access to the entire sequence of membrane potential states, which must be stored during the forward pass to compute gradients during backpropagation. This effect imposes a memory footprint that scales linearly with the length of the temporal sequence (T), the number of neurons in each layer (H), and the number of layers (L), resulting in a spatial complexity of $\mathcal{O}(THL)$. Furthermore, this non-local dependency collides with the local update mechanism observed on the brain, where each synaptic update depends exclusively on the current or recent states of neurons.

These challenges not only highlight the biological implausibility of BPTT but also limit its implementation in resource-constrained hardware accelerators. In the following sections, we will introduce how the limitations of spatial and temporal credit assignment of BPTT have been addressed in the literature.

1.3.1. Addressing limitations in temporal credit assignment

To solve the non-locality of the temporal credit assignment in BPTT, *three-factor* learning rules have been proposed [2, 7–9, 22]. This family of learning algorithms approximate the BPTT gradients as

$$\left(\frac{\partial E_L}{\partial \theta_l[i,j]}\right)_{\text{BPTT}} \approx \left(\frac{\partial E_L}{\partial \theta_l[i,j]}\right)_{\text{three-factors}} = \sum_t S_l^t[j]\, \epsilon_l^t[i,j] \tag{8}$$

where $S_l^t[j]$ is a top-down learning signal that acts as a modulatory factor, conveying spatial credit assignment information. In contrast, $\epsilon_l^t[i,j]$ is a transient synaptic variable known as the *eligibility trace*, supported by experimental findings in biological neurons [23]. It captures the history of synaptic activity through an exponential low-pass filter version of both pre- and post-synaptic activities, and locally encodes the *temporal credit assignment*. This separation allows learning to occur in a time-local and biologically plausible manner, as illustrated in figure 1. In generic form, eligibility traces are defined as

$$\epsilon_l^t[i,j] = \beta \epsilon_l^{t-1}[i,j] + g\left(s_{l-1}^t[i]\right) f(s_l^t[j]), \tag{9}$$

where β is a decay factor. The precise formulation of $g(\cdot)$, $f(\cdot)$ and the learning signal $S_l^t[j]$ vary between different rules depending on how the spatio-temporal gradients are derived and separated. For instance, the eligibility trace of E-prop [2] defines the presynaptic factor as a low-pass filtered version of the spiking activity, $g(s_{l-1}^t[i]) = \alpha g(s_{l-1}^{t-1}[i]) + s_{l-1}^t[i]$, and the postsynaptic factor as the surrogate derivative of the spike function, $f(s_l^t[j]) = \Theta'(v_l^{t+1}[j])$. The learning signal is defined as an output-based modulatory signal as $(S_l^t[j])_{\text{e-prop}} = (c_L^t[c] - c^*[c])B_l[c,j]$, with c being one of the C output neurons and B_l providing a fixed mapping that implements a structural linear dimensionality reduction from layer l to the output layer.

On the other hand, OSTL [8], provides a more accurate approximation of BPTT where eligibility traces are derived to be equivalent to BPTT for single-layer recurrent networks, while maintaining temporal locality. Moreover, differently from E-prop [2], OSTL explicitly separates the spatial and temporal components in a layer-wise manner, enabling the learning signal to be the exact spatial gradient of error backpropagation.

While these solutions offer a biologically plausible solution to the non-locality of temporal credit assignment of BPTT, enabling time-local updates without storing activations at each time step, they still suffer from a significant memory overhead.

Specifically, eligibility traces are stored per synapses, leading to a space complexity that scales quadratically with the number of neurons, i.e. $\mathcal{O}(H^2L)$, making it prohibitive for deep architectures. To reduce the quadratic complexity of standard eligibility formulations, recent learning rules such as S-TLLR [9] and OTTP [7] propose to set $\beta = 0$ in equation (9), and keep only filtered versions of the pre- and post-synaptic spiking activities. These formulations enable an update rule that depends solely on local activity trading performance for a space complexity of $\mathcal{O}(HL)$.

1.3.2. Addressing limitations in spatial credit assignment

To address the limitations of spatial credit assignment, different alternatives have been proposed in the literature, including Feedback-Alignment (FA) [19], Direct-Feedback-Alignment (DFA) [24], and Direct-Random-Target-Projection (DRTP) [14]. FA addresses the weight-transport problem by replacing $W^\top$ in equation (7) with a fixed random matrix B_l of the same dimensions. DFA builds on this idea but proposes to bypass the backward pass by directly projecting the output error gradient to each hidden layer using a fixed random matrix, resulting in the following update rule:

$$\left(\frac{\partial E_L}{\partial v_l^t[j]}\right)_{\text{DFA}} = \frac{\partial E_L}{\partial v_L^t[k]} \cdot B_l[k,j] \cdot \frac{\partial s_l^t[j]}{\partial v_l^t[j]} \tag{10}$$

This strategy is used in OSTL to derive a more biologically plausible learning rule, OSTL(rnd), which maintains temporal locality while eliminating the weight-transport problem. Nevertheless, these solutions still rely on top-down error propagation from the output layer, leaving the update-locking problem unaddressed.

To address both update-locking and weight-transport problems, DRTP was proposed in [14] as a purely feedforward alternative, where the target output ($c^*[c]$) replaces the gradient term $\partial E_L/\partial v_L^t$ in equation (10) and is directly projected to the hidden layers. This eliminates the need to wait for a top-down error propagation, as the target vector is available before the forward pass begins. This approach not only enables the pipelining of input signals, but also eliminates the need to store activations at each time step.

Nonetheless, all these approaches require an additional projection matrix for each hidden layer (i.e. B_l). While this can be sustainable in terms of storage for fully-connected architectures, it can lead to significant memory overhead in convolutional architectures, where the activations dominate the memory footprint. Thus, with the intention of removing both the backward pass and the storage overhead of auxiliary projection matrices, a novel class of learning algorithms known as *Forward-Only* [15–17] has recently emerged.

Forward-Only algorithms fully eliminate the backward pass and rely exclusively on multiple forward passes to drive learning. For example, PEPITA [17] drives its learning based solely on the activity difference between two forward passes:(1) a standard forward propagation of the input signal and (2) a forward propagation of an error-modulated version of the same input. On the other hand, Forward–Forward [15] and Signal Propagation [16] are inspired by contrastive learning to drive the parameter updates. For example, Forward–Forward optimizes each layer parameter to distinguish between *positive* (real) and *negative* (fake) inputs by maximizing a local *goodness* metric, generally the squared sum of activities, for positive examples and minimizing it for negative ones. Specifically, positive examples are input data paired with correct labels, while negative examples are corrupted input data paired with incorrect labels.

In contrast, SP [16] minimizes a layer-wise cross-entropy loss computed over pairwise similarities between input activations and target activation within the same batch. These targets are generated by projecting the desired batched output targets $c^*[c]$, through a target propagator that brings the targets to the same dimension as the first hidden layer, allowing direct comparison. Thus, by minimizing the local loss, SP maximizes the similarity between an input example activation and the target of its corresponding class, while minimizing the similarity to targets of other classes within the same batch. Furthermore, during training, the targets are also jointly optimized to move closer to the representation of their respective classes and further from those of other classes, thereby generating linearly separable representations.

Notably, SP and PEPITA require only a single additional projection matrix, used for the target propagator and for the error-based modulation factor, respectively, whereas Forward–Forward require a mechanism to construct positive and negative examples. These approaches significantly reduce the memory footprint compared to previously discussed algorithms, as no additional matrices are needed to scale to deeper architectures.

1.3.3. Towards fully-local learning in SNNs

Research for bio-inspired and hardware-efficient learning rules for training SNNs is increasingly directed toward fully local learning, as depicted in figure 1(c). These kinds of learning rules attempt to approximate gradient-based learning using only information that is locally available in time and space, thus eliminating the need for global signals, weight-transport, and storage of temporal activity. One of the first examples of this kind of learning rule is DECOLLE, proposed in [12]. It employs random readout layers that project the spiking activations s_l^t of each hidden layer onto the dimension of the true target vector $c^*[c]$, through random fixed matrices $B_l \in \mathbb{R}^{H_l \times C}$, to generate local targets c_l^t. In each layer, a local mean squared error (MSE) is computed between $c_l^t[c]$ and $c^*[c]$, and the gradient of this loss with respect to the learnable parameters provides a post-synaptic learning signal. This signal is then multiplied with a second-order low-pass filtered version of the pre-synaptic activity, forming a local three-factor learning rule that enables fully local, online updates.

Algorithms such as ETLP [10] and OSTTP [11] both employ the learning signal proposed in DRTP as a learning signal to modulate eligibility traces. Specifically, ETLP employs the eligibility traces defined in e-prop [2], while OSTTP uses the trace formulation of OSTL [8]. Nevertheless, all the aforementioned solutions have been demonstrated only for small-scale architecture. Among them, DECOLLE stands out as the only method demonstrated on a network with up to three convolutional layers, highlighting its relative scalability in the context of deep SNNs.

Recently, a novel learning algorithm known as TESS [13] has been proposed, being the first to showcase fully local learning in deep structures, scaling up to VGG-9 architectures. It builds on the same eligibility traces of [9], but introduces a novel local learning mechanism for spatial credit assignment. It works similarly to the local loss proposed by DECOLLE, but instead of computing the gradient of the MSE loss, it projects back a direct comparison between a generated target and the true target through the transposition of matrix B, resulting in a learning signal $S_l^t[j] = B_l^\top \left(f\left(B_l[c,j]s_l^t[j]\right) - c^*[c]\right)$.

Despite their differences, all the aforementioned algorithms require additional projection matrices (e.g. B_l) at each layer to solve the spatial credit assignment problem. Although this is effective and shows scalability, as in the case of TESS, it introduces a significant memory overhead, particularly in convolutional architectures where such matrices must match the number of channels and spatial dimensions. In contrast, leveraging forward-only strategies such as Signal Propagation discussed in section 1.3.2 offers a promising alternative where only one extra projection matrix is required for the first layer.

2. Results

The application of signal propagation [16] to SNNs remains unclear due to the inherent temporal dynamic of these architectures. In this work, we hypothesize that the use of eligibility traces is sufficient to learn temporal dynamics using the Signal Propagation algorithm. Inspired by its formalism, our proposed solutions aim at clustering the layer-wise activity traces of the input examples belonging to the same class close to the target traces of the same class examples and apart from target traces of examples belonging to other classes. To derive the update rule of the TP algorithm in the following section, we employ the same LIF model as defined in equations (1) and (2), and follow the previously introduced notation and assumed dimensionality.

In this section, we begin by explaining the update rule of TP in section 2.1. Following this introduction to mathematical theory, we present experimental results on conventional SNN benchmarks in section 2.2 and a comparison for hardware implementation in section 2.3. Ultimately, we provide real-world use cases for fine-tuning in section 2.

2.1. TP

At each layer l, we maintain two distinct activity traces, the input and target traces, computed as a leaky integration of the local spiking activity defined as

$$\epsilon_l^t[b,j] = \beta_l \epsilon_l^{t-1}[b,j] + s_l^t[b,j], \tag{11}$$

$$\tilde{\epsilon}_l^t[b,j] = \beta_l \tilde{\epsilon}_l^{t-1}[b,j] + \tilde{s}_l^t[b,j], \tag{12}$$

where (11) and (12) represent the trace obtained by propagating through the network the input spikes s_0^t and the one-hot encoded target vector c, respectively. In particular, in this context, we make the batch index b explicit, as it will play a fundamental role in the loss definition. For clarity, we refer to (11) as the *input trace* and to (12) as the *target trace*. The latter acts as a spatio-temporal carrier of target information, enabling global information to be propagated forward across layers in the absence of a

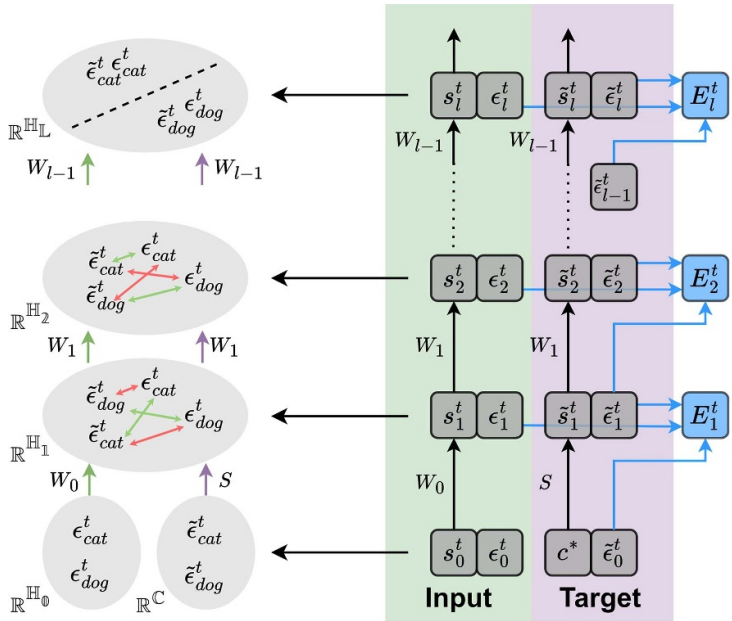

Figure 2. Overview of traces propagation. *Right*: Parallel computational paths. In the green path, the input signal s_0^t is propagated through the network, via conventional matrices W_l, generating input traces ϵ_l^t based on spiking activity s_l^t. In the purple path, the one-hot encoded target vector $c^* \in \mathcal{R}^C$ is projected to the first layer via $S \in \mathcal{R}^{C \times H_1}$, to match the dimensionality of the first input trace (i.e. ϵ_1^t), thus enabling the comparison of input and target traces through local loss E_l^t. For $l > 1$, the same matrices W_l are used to propagate the target signal and generate target traces $\tilde{\epsilon}_l^t$ at each layer. *Left*: Geometric view and trace alignment dynamics. Target traces attract input traces of the same class (green arrows) and repel those from different classes (red arrows), fostering class separation. At the last layers, the input traces of different classes become linearly separable (dashed line).

backward global error signal. At each layer, the local loss is inspired by [16] and defined as

$$E_l^t = -y_l^t[b,b'] \log\left(\mathrm{Softmax}\left(z_l^t[b,b']\right)\right) \tag{13}$$

where z_l^t and y_l^t are defined as

$$z_l^t[b,b'] = \epsilon_l^t[b,j]\,\tilde{\epsilon}_l^t[j,b'], \tag{14}$$

$$y_l^t[b,b'] = \mathrm{Softmax}\left(f\left(\tilde{\epsilon}_{l-1}^t[b,j], \tilde{\epsilon}_{l-1}^t[j,b']\right)\right). \tag{15}$$

Equation (14) defines the logits used in CE loss, and corresponds to a dot-product similarity score computed over the feature dimension. This score measures how well the trace of the input example b aligns with the target trace of the corresponding example b'. On the other hand, (15) is the target of CE loss, defined as a row-wise softmax of a generic similarity function $f(\cdot)$ (e.g. dot product or negative Euclidean distance), computed between the examples of the target traces of the previous layer. Since these target traces ultimately originate from the one-hot encoded labels, the resulting similarity matrix y_l^t imposes a class-structured geometry that is carried forward through the hierarchy. In this way, each layer receives an indirect but globally consistent coordination signal, allowing higher-level task information (i.e. the targets) to influence earlier representations in a forward-only manner and partially compensating for the lack of a global backward error. Thus, minimizing the loss (13) in each layer with respect to any learnable parameter $\theta_l = \{W_l[i,j], R_l[j,j], \beta_l, \alpha_l\}$ as per equation (16), the networks learn to move the traces of each sample close to their target traces while pushing them away from the representations of other samples, resulting in linearly separable representations at the final layer, as shown in figure 2.

$$\frac{\partial E_l^t}{\partial \theta_l} = \frac{\partial E_l^t}{\partial z_l^t}\frac{\partial z_l^t}{\partial \theta_l} \tag{16}$$

$$= \frac{\partial E_l^t}{\partial z_l^t}\left[\underbrace{\frac{\partial z_l^t}{\partial \epsilon_l^t}\frac{\partial \epsilon_l^t}{\partial s_l^t}\frac{\partial s_l^t}{\partial v_l^t}\frac{\partial v_l^t}{\partial \theta_l}}_{\text{input}} + \underbrace{\frac{\partial z_l^t}{\partial \tilde{\epsilon}_l^t}\frac{\partial \tilde{\epsilon}_l^t}{\partial \tilde{s}_l^t}\frac{\partial \tilde{s}_l^t}{\partial \tilde{v}_l^t}\frac{\partial \tilde{v}_l^t}{\partial \theta_l}}_{\text{target}}\right]. \tag{17}$$

In particular, for the synaptic connection $W_l[i,j]$, equation (17) yields the following fully local gradient:

$$\frac{\partial E_l^t}{\partial W_l[i,j]} = (\mathrm{Softmax}(z_l^t) - y_l^t)[b,b'] \left[\tilde{\epsilon}_l^t[b',j]\, \Theta'(v_l^t[b,j] - V_{\mathrm{th}})\, s_{l-1}^t[b,i] + \epsilon_l^t[b,j]\, \Theta'(\tilde{v}_l^t[b',j] - V_{\mathrm{th}})\, \tilde{s}_{l-1}^t[b',i] \right]. \tag{18}$$

In (18) the signal $(\mathrm{Softmax}(z_l^t) - y_l^t)[b,b']$ is a modulatory signal that measures how much the input trace of sample b and the target trace of sample b' should be pulled together or pushed apart, thus creating class-based clustering. This push and pull interaction is depicted by the green and red arrows, respectively, on the right side of figure 2. Notably, each synaptic update, follows a typical three-factor rule, as explained in section 1.3.1, and requires information that is local in time and space. However, due to its reliance on inter-sample contrast (i.e. $[b,b']$), this update rule cannot be computed in an online, per-sample fashion and requires a batch size of at least 2. This constraint differentiates TP from a strictly online update rule where synaptic updates occur sample by sample, reflecting a top-down design choice inspired by ANN learning principles for efficient spatial credit assignment, such as SP, and adapted to SNNs, in line with the view that neuromorphic systems can benefit from ANN-inspired engineering intuition [25]. This approach intentionally trades strict sample-wise biological learning for improved efficiency and scalability. In fact, TP does not require layer-wise projection matrices, unlike prior local learning rules, which incur a memory cost that scales with both network depth and output dimensionality; instead, TP employs a single projector together with a batch-dependent modulation term whose dimensionality can be chosen according to available hardware resources rather than the problem complexity. We provide the algorithm for the proposed update rule in appendix.

2.2. Experimental results

Following the mathematical formulation, we evaluate the effectiveness of the proposed learning rule on standard datasets for SNNs such as N-MNIST [26], SHD [27], DVS-GESTURE [28], and DVS-CIFAR10 [29]. The details of the pre-processing and the hyperparameter tuning strategy are discussed in detail in section 3.1. Specifically, we test TP in both fully connected and convolutional architectures: N-MNIST and SHD are evaluated with fully connected architectures, using recurrent connections for the case of SHD, while DVS-GESTURE and DVS-CIFAR10 are tested with VGG-9 convolutional neural networks, to demonstrate its scaling capabilities.

The results for fully connected architectures are reported in table 1, while those for convolutional architectures are reported in table 2. Remarkably, TP maintains the highest performance across all fully local algorithms for both N-MNIST and SHD. Compared to BPTT, TP loses only 1.12 percentage points (pp) on N-MNIST with feed-forward architectures and 1.43 pp on SHD with recurrent connections. On more complex datasets like DVS-GESTURE and DVS-CIFAR10, TP maintains competitive performance even in deep architectures like VGG-9 but fails to achieve the SOTA performance achieved by TESS, the only other fully local and scalable algorithm. Specifically, TP loses only 0.33 pp compared to TESS and BPTT on the DVS-GESTURE, while on the DVS-CIFAR10 dataset, TP loses 4.78 pp from BPTT 3.38 pp from TESS. We attribute this larger discrepancy to the contrastive nature of TP, which can be hindered by the high inter-class similarity of the CIFAR10 datasets [30].

2.3. Memory and computation complexity

In this section, we characterize TP in terms of its memory and computational complexity, and compare it to BPTT and other local learning rules. The results of this comparison are summarized in table 3. In this analysis, we assume an equal number of neurons H in all layers L. Like in [13], for estimating the time complexity, we exclude the computation time for the traces and the outer product with the presynaptic term, which would otherwise dominate the expression with a $\mathcal{O}(H^2)$ term and make all algorithms appear similarly expensive. Thus, the time complexity is expressed considering only the modulatory signal. In the case of TP, the modulatory signal is $(y_l^t - z_l^t)[b,b']$. The time to compute this modulatory signal is $\mathcal{O}(B^2LH)$, as it is obtained by computing pairwise similarities between all inputs and target traces at each layer for each example in the batch. Compared to other fully-local algorithms such as DECOLLE [12], ETLP [10], OSTTP [11] and TESS [13], which have a time complexity of $\mathcal{O}(BLOH)$, due to the output error or target projections to generate the modulation signal, TP trades the dependency on the number of output neurons for a factor of B^2.

Table 1. Comparison to SoTA results on N-MNIST and SHD. Bold values indicate the learning rule with the highest test accuracy among fully local rules.

Model	Architecture Type	Neuron Type	Number of Neurons	Local Learning	Time Steps	Number of Epochs	Test Accuracy
			N-MNIST				
BPTT (ours)	Feed-Forward	LIF	200	✘	10	100	98.45 ± 0.04
eProp [2]	Feed-Forward	LIF	200	Partial (time)	10	100	97.90[a]
ETLP [10]	Feed-Forward	LIF	200	✔	10	100	94.30
DECOLLE [12]	Feed-Forward	LIF	200	✔	10	100	96.27[a]
TP (ours)	**Feed-Forward**	**LIF**	**200**	✔	**10**	**100**	$\mathbf{97.33 \pm 0.06}$
			SHD				
BPTT (ours)	Feed-Forward	LIF	450	✘	100	100	75.85 ± 0.48
eProp [2]	Feed-Forward	LIF	450	Partial (time)	100	100	63.04[a]
ETLP [10]	Feed-Forward	ALIF	450	✔	100	100	59.19
DECOLLE [12]	Feed-Forward	LIF	450	✔	100	100	58.55[a]
TP (ours)	**Feed-Forward**	**LIF**	**400**	✔	**100**	**100**	$\mathbf{67.06 \pm 0.96}$
BPTT (ours)	Recurrent	LIF	450	✘	100	100	83.23 ± 1.00
S-TLLR [9]	Recurrent	LIF	450	Partial (time)	100	100	78.24 ± 1.84
eProp [2]	Recurrent	LIF	450	Partial (time)	100	100	80.79[a]
OSTTP [11]	Recurrent	SNU	450	✔	100	100	77.33 ± 0.80
ETLP [10]	Recurrent	ALIF	450	✔	100	100	74.59[a]
DECOLLE [12]	Recurrent	LIF	450	✔	100	100	62.01[a]
TP (ours)	**Recurrent**	**LIF**	**450**	✔	**100**	**100**	$\mathbf{81.80 \pm 0.51}$

[a] Results from [10].

The space complexity at each time step of all time-local learning rules is dominated by the traces storage. For instance, OSTTP [11], which is based on OSTL [8] stores traces in a per-synaptic fashion, leading to a space complexity of $\mathcal{O}(LH^2)$. On the other hand, algorithms such as ETLP [10], S-TLLR [9], DECOLLE [12] and TESS [13], store traces per neuron, leading to a reduced space complexity of $\mathcal{O}(LH)$. Similarly, as defined in equations (11) and (12), TP requires two traces that are stored per neuron, one for the input signal and another for the target signal, leading to a total memory cost of $2BLH$ and a complexity of $\mathcal{O}(LH)$. However, while TP has a comparable space complexity to other fully-local algorithms, as mentioned in section 1.3.3, these algorithms require storing auxiliary matrices B_l to perform the spatial credit assignment, resulting in an additional memory cost of LOH. On the other hand, TP requires only a single auxiliary matrix for the target propagator, reducing the memory cost to OH. A detailed explanation of the computation and memory requirements for TP is provided in section 3.3.

2.4. Use cases

In this section, we further evaluate TP in a real-world use case. Specifically, we demonstrate its ability to perform efficient on-device fine-tuning for use-specific adaptation in speech recognition.

2.4.1. Fine-tuning on GSC

In the previous section, we demonstrated the effectiveness of TP in training from scratch on standard SNN datasets. However, practical deployment on an edge device requires adaptation to new user distributions or environments rather than training from scratch. For example, in speech recognition tasks, the model is conventionally trained on a cloud server and subsequently deployed to an edge device, as depicted in figure 3(a). This process fails to account for user-specific variations, such as speaking style, accent, or ambient noise, reducing performance once deployed. Adapting the model to these variations requires collecting user-specific samples and transmitting them to the cloud for retraining, introducing challenges related to privacy, latency, and bandwidth usage.

Motivated by the efficient and fully local update rule of TP, we investigated its applicability for on-device fine-tuning using the GSC dataset. The details of the dataset pre-processing and split are discussed in detail in section 3.1. With this objective, we trained an SNN of two hidden layers with recurrent connections on all user recordings of the GSC dataset, except those of the user *c50f55b8*, which we

Table 2. Comparison to SoTA results on IBM-GESTURE and DVS-CIFAR10. Bold values indicate the learning rule with the highest test accuracy among fully local rules.

Model	Architecture Type	Neuron Type	Local Learning	Time Steps	Number of Epochs	Test Accuracy
			DVS-GESTURE			
BPTT (ours)	VGG-9	LIF	✘	20	200	98.56 ± 0.15
OTTT [7]	VGG-9	LIF	Partial (time)	20	200	96.88
S-TLLR [9]	VGG-9	LIF	Partial (time)	20	200	98.48 ± 0.37
DECOLLE [12]	3-CONV	LIF	✔	1800	200	95.54 ± 0.16
TESS [13]	**VGG-9**	**LIF**	✔	**20**	**200**	**98.56 ± 0.31**
TP(**ours**)	VGG-9	LIF	✔	20	200	98.18 ± 0.15
			DVS-CIFAR10			
BPTT [13]	VGG-9	LIF	✘	10	200	76.40 ± 0.66
OTTT [7]	VGG-9	LIF	Partial (time)	10	200	76.27 ± 0.05
S-TLLR [9]	VGG-9	LIF	Partial (time)	10	200	75.14 ± 1.37
TESS [13]	**VGG-9**	**LIF**	✔	**10**	**200**	**75.00 ± 0.65**
TP(**ours**)	VGG-9	LIF	✔	10	200	71.39 ± 0.19

Table 3. Comparison of learning algorithms. The table highlights the key issues that limit efficient hardware implementation such as update-locking, weight-transport, time locality, and space locality. Furthermore, we include the time and space complexity of each algorithm, together with extra storage required for auxiliary matrices (e.g. B_l). The variables are as follows: L is the total number of layers, H is the number of neurons, T is the sequence length, and O is the number of output classes. In bold is our proposed learning rule.

Model	Update Locking	Weight Transport	Time Local	Space Local	Space Complexity	Time Complexity	Auxiliary Matrices
BPTT	✘	✘	✘	✘	TLH	TLH^2	–
E-prop [2]	✘	✘	✔	✘	LH^2	LH^2	–
E-prop(*rnd*) [2]	✘	✔	✔	✘	LH^2	LOH	LOH
OSTL [8]	✘	✘	✔	✘	LH^2	LH^2	–
OSTL(*rnd*) [8]	✘	✔	✔	✘	LH^2	LOH	LOH
S-TLLR [9]	✘	✘	✔	✘	LH	LH^2	–
OSTTP [11]	✔	✔	✔	✔	LH^2	LOH	LOH
DECOLLE [12]	✔	✔	✔	✔	LH	LOH	LOH
ETLP [10]	✔	✔	✔	✔	LH	LOH	LOH
TESS [9]	✔	✔	✔	✔	LH	LOH	LOH
TP (ours)	✔	✔	✔	✔	LH	LH	OH

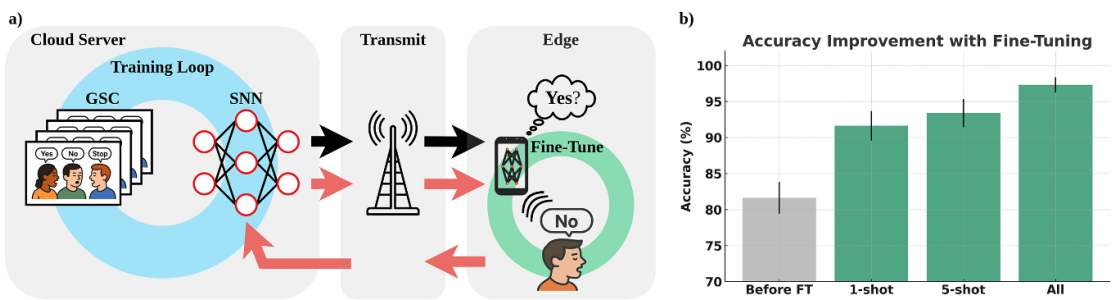

Figure 3. Google speech commands on-device fine-tuning (FT) with Traces Propagation. *a)* Edge deployment pipeline for speech recognition. A neural network is pre-trained on the Google speech command dataset in the cloud (blue circle) and transmitted to the edge device (black arrows). Without fine-tuning, the model can misclassify inputs (e.g. mistaking a 'yes' for a 'no'). If fine-tuning is not available at the edge, the new user recordings must be sent back to cloud (red arrows) for retraining, reducing privacy and increasing cost. With on-device fine-tuning (green circle) the model can adapt to the new user without data transmission. *b)* Fine tuning accuracy improvement with 1-shot, 5-shot, and all user samples. Before FT represents the accuracy on GSC for the new user following cloud-training and deployment.

reserved to simulate on-device fine-tuning. As can be seen from the gray bar in figure 3(b), which represents the accuracy of the model on the excluded user following deployment, the model correctly classifies only 81.62 % of its samples, highlighting the need for user-specific adaptation. To evaluate the capabilities of TP to perform fine-tuning, we retrained the model using a limited number of samples for 3

Table 4. Best hyperparameters and setup per dataset.

Dataset	α	β	v_{th}	Batch size
N -MNIST	0.98	0.98	1.00	128
SHD	0.97	0.96	1.00	128
SHD (rec.)	0.85	0.85	0.50	128
DVS -GESTURE	0.98	0.53	1.00	64
DVS -CIFAR10	0.19	0.18	0.50	64
GSC (rec.)	0.77	0.98	0.66	1024

epochs. Specifically, we perform 1-shot, 5-shot and all-shot fine-tuning, where only 1, 5, or all available recordings per class of the excluded user are used for retraining the model. As shown in figure 3(b), TP achieves significant improvement in accuracy, providing +10 pp, +11.76 pp and +15.68 pp for 1-shot, 5-shot, and all-shot, respectively.

Furthermore, we evaluated the ability of the model to learn solely from the user-specific samples without pre-training on other user data. In this case, performance remained low across all cases ($10.14 \pm 4.73\%$ for 1-shot, $12.84 \pm 0.68\%$ for 5-shot, and $12.84 \pm 4.73\%$ for all-shot), highlighting the importance of initial pre-training. In addition, we measured the amount of performance drop on the pre-trained model after fine-tuning, to measure the level of *catastrophic forgetting*. Specifically, the test accuracy decreased from $79.67 \pm 0.07\%$ before fine-tuning to $74.90 \pm 0.25\%$, $73.97 \pm 0.36\%$, and $74.75 \pm 0.32\%$ for 1-shot, 5-shot, and all-shot settings, respectively.

3. Materials and methods

3.1. Setup and datasets

In this section, we describe the datasets used in our experiments, together with the pre-processing steps applied to each. For each dataset, we performed hyperparameter tuning on the membrane potential decay constant α, the trace integration constant β, and the threshold v_{th}, which was selected from the values 0.5 and 1.0. The results of the hyperparameter search are summarized in table 4. All different architectures used in our experiment use a simple integrator as an output layer, where the predicted class corresponds to the neuron with the highest integration value at the end of the sequence. For all datasets, we employ a learning rate of $1e^{-4}$ and apply the layer update rule described in equation (18) at each time step. As a surrogate function θ', we employ the ArcTan function defined in [31] with a scale factor of 1. Following the hyperparameter tuning strategy, we train the network for 10 random seeds. Similarly to other works [10, 11, 13], and report the best test accuracy encounter across the whole training process. Specifically, we report the mean and standard deviation of the peak test accuracy across the top 5 seeds. We also report the architectural configurations used in each case. All experiments and implementation details can be found in the official repository[1] of this work.

3.1.1. N-MNIST

This dataset is the neuromorphic version of the conventional MNIST dataset used to benchmark ANNs. It is created by recording the MNIST digits with a Dynamic Vision Sensor (DVS) moved in three different saccades to mimic the behavior of the human eye. Similarly to [10], we retain only the first saccade of the recording and convert the event streams into frame-based tensors of 100 ms with a time step of 1 ms. From an architectural standpoint, we used a single hidden layer of 200 neurons without recurrent connection. We find the best hyperparameters to be $\beta = 0.98$, $\alpha = 0.98$, and $v_{\text{th}} = 1.0$.

3.1.2. SHD

The SHDs dataset contains recordings of spoken digits from 1 to 10, in both English and German, converted into spike trains using a biologically inspired cochlear model. The objective of the dataset is to classify the digit that is spoken. Unlike N-MNIST, this dataset is characterized by a strong temporal structure, which makes it particularly suitable for evaluating the ability of learning algorithms to assign credit over time in SNNs. To convert the events stream into frame-based tensors, we accumulate the events in a time window of 10 ms and use the entire recoding duration. From an architectural standpoint, we used a single hidden layer of 450 neurons, with and without recurrent connections. We find the best hyperparameters to be $\beta = 0.96$, $\alpha = 0.97$, and $v_{\text{th}} = 1.0$, for the feed-forward architecture, and $\beta = 0.85$, $\alpha = 0.85$, and $v_{\text{th}} = 0.5$ for the recurrent one.

[1] https://github.com/lollopes/traces_propagation.

3.1.3. DVS-GESTURE

This dataset consists of 11 hand gestures from 29 subjects under 3 illumination conditions recorded using a DVS camera. The goal of this task is to correctly classify each of the 11 hand gestures. We follow the same pre-processing steps as in [13]. Specifically, sequences of different lengths are split into samples of 1.5 s, and events are accumulated using a time window of 75 ms, leading to frame tensors of 20 time steps. Furthermore, each frame is resized to 32 × 32 and randomly cropped with a zero-padding of 4. From an architectural stand point, we employ the same VGG-9 structure as [13], to guarantee a fair comparison. Specifically, the architecture consists of 8 LIF based convolutional layers with 3 × 3 kernels, where the number of channels increases from 64 to 512, and max pooling is applied every 2 players. To stabilize the firing behavior across the whole network, weight normalization is applied to the input currents of each layer. We find the best hyperparameters to be $\beta = 0.53$, $\alpha = 0.98$, and $v_{\text{th}} = 1.0$.

3.1.4. DVS-CIFAR10

This dataset is the neuromorphic version of the conventional CIFAR10 used to benchmark CNNs. The conversion from static images to event-based streams was achieved by repeated closed-loop movement of the static image in front of a stationary DVS camera. Like in [13], events are accumulated in 10 times steps and resized to 48 × 48. For the training set, the images are randomly cropped with zero-padding of 4. The same VGG-9 architecture explained in the previous section is used for this dataset. We find the best hyperparameters to be $\beta = 0.18$, $\alpha = 0.19$, and $v_{\text{th}} = 0.5$.

3.1.5. GSC

The GSC dataset is a common benchmark for keyword spotting. It comprises 65 000 one-second WAV audio clips of 30 short English words. To convert WAV audio recordings into a spike-based signal, each audio recording is sampled at 16 kHz and padded with zero if its duration is shorter than 1 s. The resulting signal is then segmented into overlapping windows of 30 ms with a hop of 10 ms. To each window, a Fast Fourier Transform (FFT) is applied to extract its frequency components, which are then projected onto 40 mel-frequency bands distributed between 20 Hz and 4 kHz, to mimic the human auditory perception. This processing generates a time-frequency representation of 101 frames per recording. Additionally, delta and delta-delta features are computed and added as extra channels. Finally, the resulting tensors are normalized and comprise 3 channels, 101 time steps, and 40 features.

For the fine-tuning experiments outlined in section 2.4.1, we generate three sets: a training set, used to simulate cloud learning, multiple k-shot support sets for user-specific adaptation, and a query set used to evaluate the performance on the user after fine-tuning. Specifically, the training set comprises all samples from the training split of the original GSC dataset, excluding those of the user *c50f55b8*, who has the greatest number of recordings. The samples of this user are then split with an 80–20 ratio into support and query sets, respectively, resulting in 242 support samples and 74 query samples. From the support set, we create multiple k-shot subsets by sampling k examples per class, where $k \in \{1, 5, 242\}$. These k-shot support sets are used to perform the fine-tuning on user-specific data, while the query set is used to evaluate the performance of the tuned model to unseen words of the user.

For our experiment, we trained an SNN with one hidden layer of 450 neurons with recurrent connection followed by a linear integrator as the classification layer using the TP learning rule. To find the best hyperparameter we performed a hyperparameter optimization and found the best parameters to be $v_{\text{th}} = 0.66$, $\beta = 0.98$, and $\alpha = 0.77$. We train on the samples from the train set for 8 epochs using a batch size of 1024, and perform the fine-tuning stages for the different k-shots for only 3 epochs, to emulate on-device adaptation.

3.2. Characterization of trace propagation

As explained in section 2.1, the main objective of TP is to group the input traces into class-specific clusters. To validate this principle and provide an intuitive representation of this effect, we show in figure 4(a) t-SNE [33] visualization of the input traces at the end of training for each convolutional layer of the VGG-9 architecture trained on the DVS-GESTURE dataset. Specifically, we perform the t-SNE analysis using the last time step values of the input traces, as they represent a summary of the neuron activity across the whole sequence that contributed to the network's final classification. Specifically, we collect the traces of each example across the last epoch and perform a t-SNE analysis at each layer sampling only 2000 examples. From the figure, it can be observed that in the first two convolutional layers (panels 1 and 2), the 2D representations of the traces for the 11 classes are still mixed together. A clearer separation emerges in the third and fourth layers (panels 3 and 4). While distinct clusters emerge, the radius of the cluster remains significantly large, with some overlap between classes. On the other

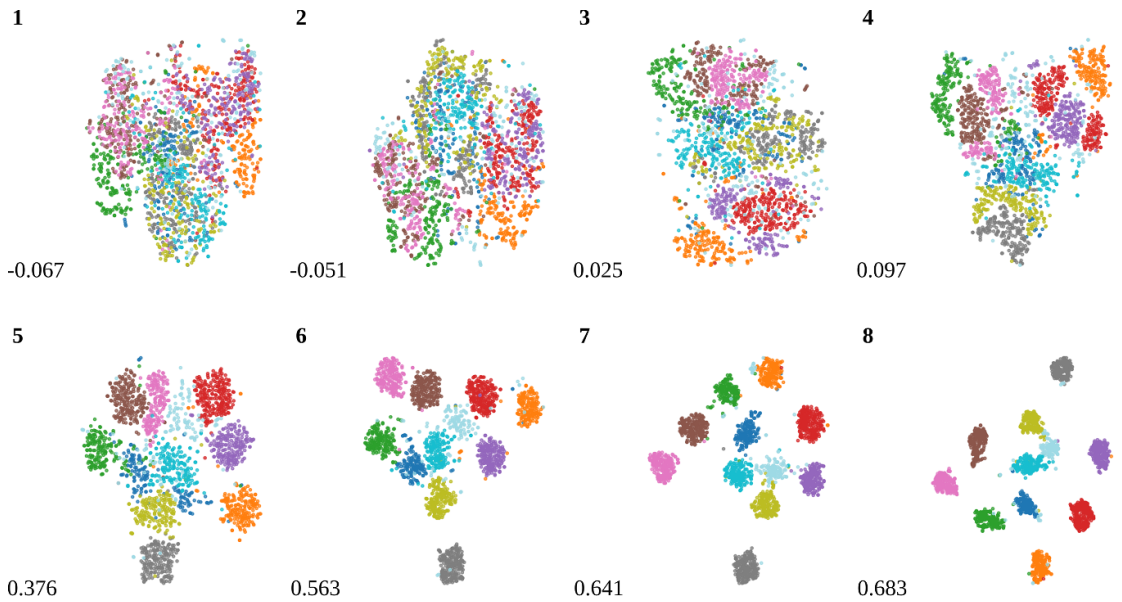

Figure 4. t-SNE visualization of input traces across layers on the DVS-GESTURE dataset. Panels 1 to 8 show the 2D representation at the end of training of the input traces for each layer of a VGG-9 architecture trained on the DVS-GESTURE dataset. In the bottom-left corner of each panel, we report the Silhouette score [32] to quantify the degree of clustering. A value of −1 indicates overlapping or poorly separated clusters, while a value of +1 corresponds to more compact and well-separated groups. As we can see, the Silhouette score increases progressively with layer depth, demonstrating that the network develops increasingly discriminative and separable representations.

hand, for layers 5 to 8, the traces start to reduce their radius and are pushed further apart in the feature space, indicating linear separability between classes.

In addition to the t-SNE visualization, we provide a characterization of the convergence behavior of TP compared to BPTT in figure 5(a). From this figure we can observe that while TP converges more slowly than BPTT in early epochs, both algorithms reach stable final training accuracy, although TP plateaus at slightly lower values than BPTT, consistent with the absence of an exact global error signal. Furthermore, even if the interaction between distant layers is indirect, increasing the network depth from three (VGG-4) to eight (VGG-9) convolutional layers improves TP performance, narrowing the performance gap with BPTT and demonstrating that TP benefits from increased network depth. To complement these results, figure 5(b) presents a qualitative comparison of the extracted feature maps for the first three convolutional layer of a VGG-9 network trained on the DVS-GESTURE dataset, providing additional insight into the learned representations. In the first convolutional layer, both TP and BPTT preserve the coarse spatial structure of the input and exhibit activation patterns that reflect the full motion of the gesture. As depth increases, the features evolve into more structured and abstract representations. For instance, in the second convolutional layer, both methods compress the gesture into smoother, blob-like activations that emphasize the dominant motion envelope, with BPTT producing slightly more compact and sparse responses than TP. By the third convolutional layer, the feature maps become sparse and strongly gesture-specific, with activity restricted to a few disjoint regions whose spatial arrangement differs between the RIGHT ARM CW and LEFT ARM CW examples. These results suggests that TP performance benefits from deeper architectures and learns hierarchical, class-discriminative features comparable to those obtained with BPTT.

3.3. Multiply-accumulate operations (MACs) and memory estimation for TESS and TP

In this section, we calculate the total memory cost and the total MACs for both TESS and TP. The total MACs of TP are found as follows:

$$\mathrm{MACs}_{\mathrm{TP}} = T\sum_{l=0}^{L}\left[\underbrace{2BH_l}_{(11)\&(12)} + \underbrace{B^2H_l}_{(14)} + \underbrace{B^2H_{l-1}}_{(15)} + \underbrace{2B^2H_{l-1}H_l}_{(18)}\right]. \tag{19}$$

In [13], the authors provide an estimate of the MACs, but consider only the modulation signal, leaving out the computational cost of the traces. Here, we want to fully characterize the computational cost

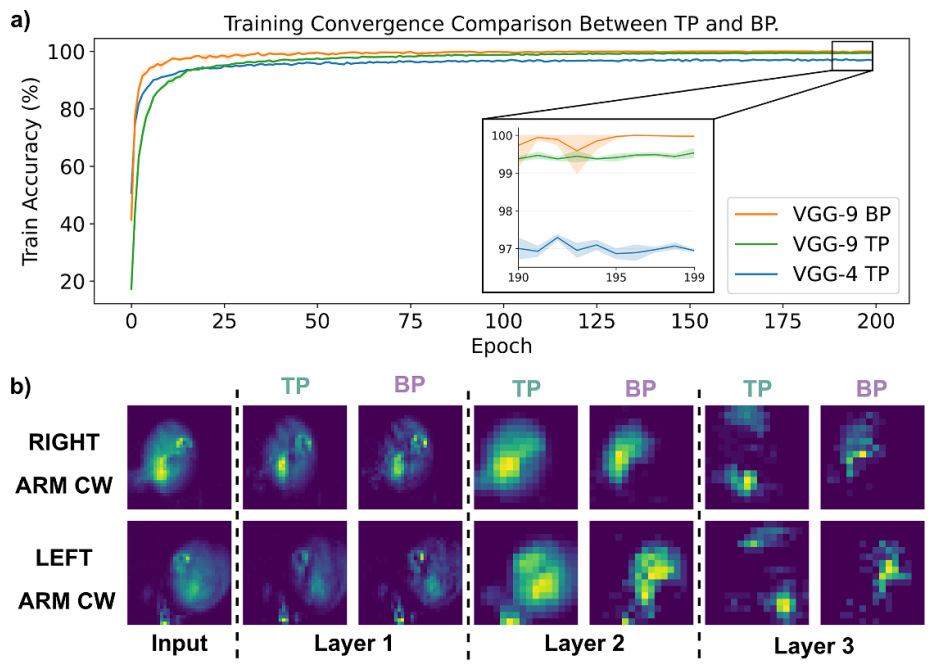

Figure 5. Comparison of training convergence and feature hierarchies between TP and BP (BPTT) on the DVS-GESTURE dataset. (a) Mean training accuracy across epochs for 5 different seeds. The shadows indicate the standard deviation across seeds. The inset contains the convergence behavior for the last 10 epochs. (b) Extracted feature maps for the first three convolutional layers of the VGG-9 model for both BPTT and TP, where the feature maps correspond to the mean spike-rate activity over the full gesture sequence. Each row corresponds to a different gesture class, namely RIGHT ARM CW and LEFT ARM CW, which denote clockwise circular arm movements performed with the right and left arm, respectively.

and include all necessary signals. To estimate the total MACs we refer to the equations numbers of the original work [13]. The total MACs for TESS can be estimated according to equation (20), where t_l represents the time-step at which the update is performed,

$$\text{MACs}_{\text{TESS}} = (T - t_l)\sum_{l=0}^{L}\left[\underbrace{BH_l}_{(7)} + \underbrace{BH_{l-1}}_{(5)} + \underbrace{BH_lO + BH_l}_{(9)} + \underbrace{2BH_{l-1}H_l}_{(11)\&(12)}\right] \tag{20}$$

On the other hand, the memory estimations for TESS and TP are calculated according to equations (21) and (23), respectively,

$$\text{Memory}_{\text{TP}} = \underbrace{OH_0}_{\textit{Auxiliary matrix S}} + \sum_{l=0}^{L}\left[\underbrace{2BH_l}_{\textit{input \& target potential}} + \underbrace{2BH_l}_{\textit{input \& target trace}}\right] \tag{21}$$

$$= OH_0 + 4B\sum_{l=0}^{L} H_l \tag{22}$$

$$\text{Memory}_{\text{TESS}} = \sum_{l=0}^{L}\left[\underbrace{BH_l}_{\textit{input potential}} + \underbrace{2BH_l}_{\textit{traces}} + \underbrace{OH_l}_{\textit{Auxiliary matrices } B_l}\right] \tag{23}$$

$$= (3B + O)\sum_{l=0}^{L} H_l. \tag{24}$$

For deep networks, the term $\sum_{l=0}^{L} H_l$ dominates H_0. Thus, we can approximate (22) as $4B\sum_{l=0}^{L} H_l$. Taking the ratio of memory estimations for TESS and TP under this assumption, we express the relative memory cost of TESS with respect to TP as shown in equation (25). From this equation, we can deduce

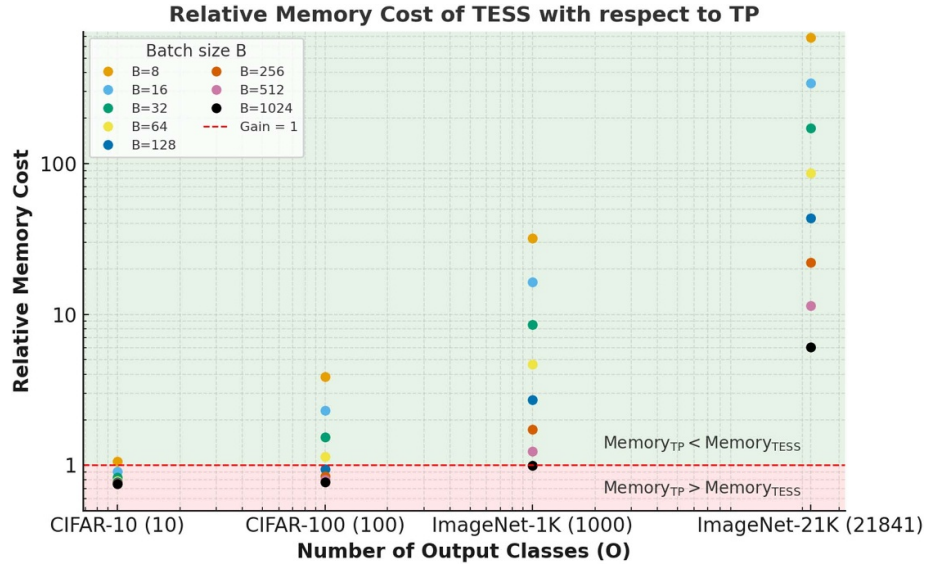

Figure 6. Relative memory cost of TP compared to TESS as a function of the number of output classes and batch size. A relative memory cost of G indicates that TESS uses G times more memory than TP.

that TP is more memory efficient whenever the number of output classes is greater than the batch size ($O > B$). For complex datasets, where the number of output classes dominates the batch size ($O >> B$), TP provides a memory gain that scales with $O/4B$, indicating that the memory advantage of TP over TESS increases linearly with the number of output classes.

$$\text{Relative Memory Cost} = \frac{\text{Memory}_{\text{TESS}}}{\text{Memory}_{\text{TP}}} \approx \frac{3B + O}{4B}. \tag{25}$$

To provide an intuitive understanding of how the memory of TESS scales compared to that of TP, we plot equation (25) in figure 6 for conventional batch sizes and a set of widely-used image classification datasets, arranged in order of increasing number of output classes: CIFAR-10, CIFAR-100, ImageNet-1 K, and ImageNet-21 K.

4. Discussions and conclusions

The on-device adaptation required to deploy deep SNNs in real-world use cases, where the environment dynamics are constantly changing, is challenged by the spatial and temporal non-locality of BPTT. To address this challenge, we propose TP, a forward-only, scalable learning rule that is local in time and space. It combines the bio-inspired mechanism of eligibility traces for local temporal credit assignment, with a layer-wise contrastive loss inspired by the Signal Propagation algorithm to address the spatial credit assignment problem.

Unlike previous scalable solutions such as TESS, TP removes the need for layer-wise auxiliary matrices at the cost of doubling the internal states, providing a favorable memory cost compared to TESS when $O > B$. In particular, TP provides a memory advantage over TESS that increases linearly with the number of output classes, while maintaining a space complexity of $\mathcal{O}(LH)$ as other state-of-the-art local learning rules. From a computational perspective, TP scales with a complexity of $\mathcal{O}(B^2LH)$, removing the dependency on the number of output classes O present in other solutions, making it advantageous for complex datasets where $O \gg B$ (i.e. the number of output classes is much greater than the batch size).

Across standard neuromorphic benchmarks, TP achieves state-of-the-art results on the N-MNIST and SHD dataset compared to other fully-local methods, with only 1.4 percentage point loss compared to BPTT. In deep architectures such as VGG-9, TP achieves a performance comparable to that of TESS and BPTT on DVS-GESTURE, falling short of only 0.33 pp, while incurring a loss of 3.38 pp relative to TESS on DVS-CIFAR10. These results demonstrate that the proposed approach is sufficient to solve

complex spatio-temporal tasks using deep architectures but can be hindered by datasets with high inter-class overlap, such as DVS-CIFAR10 [30]. Furthermore, we showcased the applicability of TP to practical use cases, such as fine-tuning on the GSC dataset, where TP improved user-specific accuracy by up to +11.76, +15.68, and +15.68 percentage points for 1-shot, 5-shot, and all-shot settings, respectively.

Although TP provides competitive performance through a learning rule that is local in space and time, showcasing scalability to deep spiking architectures across datasets of varying complexity and use cases, several limitations remain. First, the contrastive nature of TP prevents weight updates from occurring in an online per-sample fashion, making it advantageous to other solutions only when the batch size is smaller compared to the number of output classes. Second, the calculation of pairwise similarities between all examples in a given batch introduces a computational cost of B^2LH, which can hinder its applicability in resource constrained devices for large batch sizes. Third, we suspect that the contrastive nature of TP limits its performance in datasets with substantial inter-class similarity, such as CIFAR-10 [30], from which the DVS version is derived.

Future extensions of this work include the investigation of the performance of TP in quantization-aware training to enable lightweight training on embedded devices, implementing TP on conventional SNN accelerators, and extending it to continuous learning settings where new output classes can be learned directly after deployment. In addition, we plan to extend the evaluation of TP beyond the conventional keyword and gesture datasets to more diverse benchmarks, including event-camera object detection, motor prediction, and time-series forecasting, in line with the direction established by the neuromorphic community [34]. Together, these directions highlight TP as a promising step toward enabling memory-efficient, adaptive, and scalable SNNs for real-world edge intelligence.

Data availability statement

No new data were created or analysed in this study.

Acknowledgment

This publication is funded in part by the Project NL-ECO: Netherlands Initiative for Energy-Efficient Computing (with Project Number NWA. 1389.20.140) of the NWA research programme Research Along Routes by Consortia which is financed by the Dutch Research Council (NWO). This research was also partially funded by the Dutch Organization for Scientific Research (NWO) through the Self-Healing Neuromorphic Systems (SNS) Project (KICH1.ST04.22.021).

Appendix

Algorithm 1. Trace propagation algorithm.

# Requires batch size $B \geqslant 2$ due to contrastive loss
for each time step t **do**
  for each sample $b = 1, \ldots, B$ **do**
    # Input and target trace update (per sample)
    $\epsilon_0^t = \beta_0 \epsilon_0^{t-1} + s_0^t$
    $\tilde{\epsilon}_0^t = \beta_0 \tilde{\epsilon}_0^{t-1} + c$
    for each layer $l = 1, \ldots, L$ **do**
      # Input propagation
      $v_l^t = \alpha_l v_l^{t-1} + s_{l-1}^t W_l - s_l^{t-1} v_{l,th}$
      $s_l^t = \Theta(v_l^t - v_{l,th})$
      $\epsilon_l^t = \beta_l \epsilon_l^{t-1} + s_l^t$
      if $l == 1$ **then**
        $\tilde{s}_0^t = c$
        $W_l = S$
      end if
      # Target propagation
      $\tilde{v}_l^t = \alpha_l \tilde{v}_l^{t-1} + \tilde{s}_{l-1}^t W_l - \tilde{s}_l^{t-1} v_{l,th}$
      $\tilde{s}_l^t = \Theta(\tilde{v}_l^t - v_{l,th})$
      $\tilde{\epsilon}_l^t = \beta_l \tilde{\epsilon}_l^{t-1} + \tilde{s}_l^t$
    end for
  end for
  for each layer $l = 1, \ldots, L$ **do**
    # Contrastive similarity and synaptic update (batch-wise)
    $z_l^t = \epsilon_l^t (\tilde{\epsilon}_l^t)^\top$
    $y_l^t = \mathrm{Softmax}\left(-f_{\mathrm{sim}}(\tilde{\epsilon}_{l-1}^t, \tilde{\epsilon}_{l-1}^t)\right)$
    $\Delta W_{l,\mathrm{in}}^t = (Softmax(z_l^t) - y_l^t)\, \tilde{\epsilon}_l^t \Theta'(v_l^t - V_{\mathrm{th}})\, s_{l-1}^t$
    $\Delta W_{l,\mathrm{trg}}^t = (Softmax(z_l^t) - y_l^t)\, \epsilon_l^t \Theta'(\tilde{v}_l^t - V_{\mathrm{th}})\, \tilde{s}_{l-1}^t$
    $W_l^{t+1} = W_l^t - \eta \left(\Delta W_{l,\mathrm{in}}^t + \Delta W_{l,\mathrm{trg}}^t\right)$
  end for
end for

ORCID iDs

Lorenzo Pes 0000-0002-8151-9327
Bojian Yin 0000-0002-5074-4337
Sander Stuijk 0000-0002-2518-6847
Federico Corradi 0000-0002-5868-8077

References

[1] Lillicrap T P, Santoro A, Marris L, Akerman C J and Hinton G 2020 Backpropagation and the brain *Nat. Rev. Neurosci.* **21** 335–46
[2] Bellec G, Scherr F, Subramoney A, Hajek E, Salaj D, Legenstein R and Maass W 2020 A solution to the learning dilemma for recurrent networks of spiking neurons *Nat. Commun.* **11** 3625
[3] Eshraghian J K *et al* 2023 Training spiking neural networks using lessons from deep learning *Proc. IEEE* **111** 1016–54
[4] Czarnecki W M, Świrszcz G, Jaderberg M, Osindero S, Vinyals O and Kavukcuoglu K 2017 Understanding synthetic gradients and decoupled neural interfaces *Int. Conf. on Machine Learning* vol 70 (PMLR) pp 904–12
[5] Grossberg S 1987 Competitive learning: from interactive activation to adaptive resonance *Cogn. Sci.* **11** 23–63
[6] Werbos P J 2002 Backpropagation through time: what it does and how to do it *Proc. IEEE* **78** 1550–60
[7] Xiao M, Meng Q, Zhang Z, He D and Lin Z 2022 Online training through time for spiking neural networks *Advances in Neural Information Processing Systems* vol 35 pp 20717–30
[8] Bohnstingl T, Woźniak S, Pantazi A and Eleftheriou E 2022 Online spatio-temporal learning in deep neural networks *IEEE Trans. Neural Netw. Learn. Syst.* **34** 8894–908
[9] Apolinario M P E and Roy K 2025 S-TLLR: STDP-inspired temporal local learning rule for spiking neural networks *Trans. Mach. Learn. Res.*
[10] Quintana F M, Perez-Peña F, Galindo P L, Neftci E O, Chicca E and Khacef L 2024 Etlp: event-based three-factor local plasticity for online learning with neuromorphic hardware *Neuromorph. Comput. Eng.* **4** 034006
[11] Ortner T, Pes L, Gentinetta J, Frenkel C and Pantazi A 2023 Online spatio-temporal learning with target projection *2023 IEEE 5th Int. Conf. on Artificial Intelligence Circuits and Systems (AICAS)* pp 1–5
[12] Kaiser J, Mostafa H and Neftci E 2020 Synaptic plasticity dynamics for deep continuous local learning (decolle) *Front. Neurosci.* **14** 424

[13] Apolinario M P E, Roy K and Frenkel C 2025 Tess: a scalable temporally and spatially local learning rule for spiking neural networks *2025 Int. Joint Conf. on Neural Networks (IJCNN)* pp 1–9
[14] Frenkel C, Lefebvre M and Bol D 2021 Learning without feedback: fixed random learning signals allow for feedforward training of deep neural networks *Front. Neurosci.* **15** 629892
[15] Hinton G 2022 The forward-forward algorithm: some preliminary investigations (arXiv:2212.13345)
[16] Kohan A, Rietman E A and Siegelmann H T 2024 Signal propagation: the framework for learning and inference in a forward pass *IEEE Trans. Neural Netw. Learn. Syst.* **35** 8585–96
[17] Dellaferrera G and Kreiman G 2022 Error-driven input modulation: solving the credit assignment problem without a backward pass *Int. Conf. on Machine Learning* (PMLR) pp 4937–55
[18] Burkitt A N 2006 A review of the integrate-and-fire neuron model: I. Homogeneous synaptic input *Biol. Cybern.* **95** 1–19
[19] Lillicrap T P, Cownden D, Tweed D B and Akerman C J 2016 Random synaptic feedback weights support error backpropagation for deep learning *Nat. Commun.* **7** 13276
[20] Tsifakis D, Rendell A P and Strazdins P E 2004 Cache oblivious matrix transposition: simulation and experiment *Int. Conf. on Computational Science* (Springer) pp 17–25
[21] Lopes G, Fernandes J P and Paquete L 2019 An empirical study on the energy efficiency of matrix transposition algorithms *Central European Functional Programming School* (Springer) pp 375–91
[22] Zenke F and Ganguli S 2018 Superspike: supervised learning in multilayer spiking neural networks *Neural Comput.* **30** 1514–41
[23] Gerstner W, Lehmann M, Liakoni V, Corneil D and Brea J 2018 Eligibility traces and plasticity on behavioral time scales: experimental support of neohebbian three-factor learning rules *Front. Neural Circuits* **12** 53
[24] Nøkland A 2016 Direct feedback alignment provides learning in deep neural networks *Advances in Neural Information Processing Systems* vol 29
[25] Frenkel C, Bol D and Indiveri G 2023 Bottom-up and top-down approaches for the design of neuromorphic processing systems: tradeoffs and synergies between natural and artificial intelligence *Proc. IEEE* **111** 623–52
[26] Orchard G, Jayawant A, Cohen G K and Thakor N 2015 Converting static image datasets to spiking neuromorphic datasets using saccades *Front. Neurosci.* **9** 437
[27] Cramer B, Stradmann Y, Schemmel J and Zenke F 2020 The Heidelberg spiking data sets for the systematic evaluation of spiking neural networks *IEEE Trans. Neural Netw. Learn. Syst.* **33** 2744–57
[28] Amir A *et al* 2017 A low power, fully event-based gesture recognition system *Proc. IEEE Conf. on Computer Vision and Pattern Recognition* pp 7243–52
[29] Li H, Liu H, Ji X, Li G and Shi L 2017 Cifar10-dvs: an event-stream dataset for object classification *Front. Neurosci.* **11** 244131
[30] Murthy V N, Singh V, Chen T, Manmatha R and Comaniciu D 2016 Deep decision network for multi-class image classification *Proc. IEEE Conf. on Computer Vision and Pattern Recognition* pp 2240–8
[31] Fang W, Yu Z, Chen Y, Masquelier T, Huang T and Tian Y 2021 Incorporating learnable membrane time constant to enhance learning of spiking neural networks *Proc. IEEE/CVF Int. Conf. on Computer Vision* pp 2641–51
[32] Shahapure K R and Nicholas C 2020 Cluster quality analysis using silhouette score *2020 IEEE 7th Int. Conf. on Data Science and Advanced Analytics (DSAA)* (IEEE) pp 747–8
[33] Maaten v d L and Hinton G 2008 Visualizing data using t-sne *J. Mach. Learn. Res.* **9** 2579–605
[34] Yik J *et al* 2025 The neurobench framework for benchmarking neuromorphic computing algorithms and systems *Nat. Commun.* **16** 1545